

NATIONAL RESEARCH UNIVERSITY
HIGHER SCHOOL OF ECONOMICS

Faculty of Computer Science
Bachelor's Programme "Applied Mathematics and Informatics"

Research Project Report on the Topic:
Speech Enhancement Algorithms for Low-Resource Devices

Submitted by the Student:

group #БПМИ231, 3rd year of study

Larin Ilya Aleksandrovich

Approved by the Project Supervisor:

Kaledin Maksim Lvovich

Research Fellow

Faculty of Computer Science, HSE University

Contents

Annotation	3
1 Introduction	4
2 Related Work	5
2.1 Lightweight Speech Enhancement	5
2.2 DeepFilterNet	5
2.3 Architectural Extensions	6
2.4 Research Gap	6
3 Methodology	7
3.1 Signal model	7
3.2 Original DeepFilterNet architecture	7
3.3 FastGRNN block	8
3.4 Convolutional blocks	10
3.5 Affine PReLU	10
3.6 Loss	11
3.7 Evaluation metrics	12
4 Experimental setup	13
4.1 Datasets	13
4.2 Baselines	13
4.3 Training	15
4.4 Results	15
4.4.1 Our models	15
4.4.2 Comparison with other models	17
5 Plan of further work	18
6 Conclusion	18
References	20

Annotation

Deep neural network-based solutions have been widely adopted in various audio processing applications. In particular, they are used in speech enhancement, where the goal is to remove background noise and enhance the target speech signal. A key practical challenge of neural approaches is their high computational and memory cost, which becomes especially important under real-time constraints.

In this work, we explore the use of FastGRNN blocks, which allow us to reduce the number of parameters by 40% while maintaining similar performance. We also experiment with redistributing parameters and test different convolutional blocks, including variants with the Affine PReLU activation.

Аннотация

Решения на основе глубоких нейронных сетей уже достаточно давно используются в различных приложениях, связанных с обработкой звука. В частности, они применяются в задаче усиления речи (Speech Enhancement), где требуется убрать шум из аудио и усилить основной речевой сигнал. Актуальной проблемой при использовании нейросетевых решений на практике являются большие требования к вычислительным ресурсам и памяти, что вместе с необходимостью работы в реальном времени становится серьёзным ограничением.

В данной работе мы рассматриваем использование FastGRNN блоков, что позволяет уменьшить число параметров примерно на 40% при сохранении близкого качества. Также мы исследуем перераспределение параметров между частями модели и тестируем различные сверточные блоки, включая варианты с функцией активации Affine PReLU.

Keywords

DL, Speech Enhancement, Noise Reduction, Deep Filtering

1 Introduction

Speech enhancement is a well-known problem in audio signal processing. Its goal is to improve the quality and intelligibility of speech signals degraded by noise, reverberation, or other distortions. In practice, speech enhancement methods should operate in real time with low latency on CPUs or low-power devices, such as hearing aids and mobile phones. This requirement favors algorithms with a small memory footprint, which is why many works focus on reducing the parameter count. Latency is another crucial factor, especially in interactive scenarios, because large delays are noticeable and degrade user experience; therefore, it also constrains architectural choices. Computational cost is also important, since the method must run on limited hardware

Common approaches include traditional signal processing methods and, more recently, deep learning-based techniques. Traditional methods such as spectral subtraction, Wiener filtering, and statistical model-based approaches have been widely used for speech enhancement [3]. Today, these techniques are often used as components of more complex systems, which we discuss later.

Deep learning approaches have significantly improved the quality of speech enhancement. However, many of them rely on large models that are difficult to deploy on resource-constrained devices. This makes the problem of building efficient models especially important.

One of the successful approaches in this direction is DeepFilterNet [11], which combines a compact neural network with a lightweight filtering stage. It provides a good balance between quality and efficiency and serves as a strong baseline for real-time speech enhancement.

At the same time, further reduction of model complexity remains an open problem. Most existing works focus on modifying the overall architecture or improving the training procedure. In contrast, the efficiency of recurrent blocks inside such models and the distribution of parameters between different components are less explored.

In this work, we study how to reduce the complexity of DeepFilterNet while keeping the performance at a similar level. We consider two directions: replacing GRU blocks with more parameter-efficient alternatives and rebalancing parameters between the encoder-decoder and the deep filtering components.

The main contributions of this work are as follows:

- We implement the DeepFilterNet model in PyTorch and reproduce its training pipeline on the VCTK+DEMAND dataset [14].
- We evaluate the use of ComFi-FastGRNN [2] (later we name it FastGRNN for simplicity) as a replacement for GRU blocks and show that it significantly reduces the number of parameters

with only a small drop in performance.

- We study the effect of parameter rebalancing and show that the original configuration of DeepFilterNet is close to optimal in terms of quality.
- We analyze the trade-off between model quality and computational efficiency across different architectural modifications.

Our implementation is available at https://github.com/runtime57/speech_enhancement. This research was supported in part through computational resources of HPC facilities at HSE University

2 Related Work

2.1 Lightweight Speech Enhancement

A widely used lightweight baseline in speech enhancement is RNNoise [15], proposed by Valin. It partitions the spectrum into Bark-scale bands and uses a small recurrent network to estimate an ideal ratio mask (IRM) for each band. In addition, it applies a pitch-filtering module to suppress noise between harmonics.

The method is highly efficient in terms of computational cost and memory footprint, but it is typically outperformed by modern approaches on quality metrics. Nevertheless, recurrent architectures remain widely used in speech enhancement, as they are well suited for modeling sequential signals such as audio.

A summary of the discussed models and their main characteristics is shown in Table 2.1.

2.2 DeepFilterNet

Lightweight hierarchical RNNs are a main focus of Schröter et al. [11]. The authors introduce DeepFilterNet, an efficient real-time speech enhancement framework.

DeepFilterNet operates in the STFT (time–frequency) domain and uses a compact neural backbone to extract spectral features and predict a suppression mask that attenuates noise across time and frequency. To improve quality without a large increase in computation, the model also predicts coefficients for a lightweight learnable complex filtering step (“deep filtering”), which refines the enhanced spectrum and reduces residual artifacts. The enhanced waveform is reconstructed via the inverse STFT.

DeepFilterNet has become an influential solution and inspired several follow-up works. For example, DeepFilterNet2 [16] improves the original model mainly through better training procedures and data augmentation. Another extension targets hearing aid applications and studies the integration of classical filters such as Wiener and MVDR into the framework.

2.3 Architectural Extensions

Several works explore modifications of the DeepFilterNet architecture. One example is DPDFNet [6], where the authors replace standard RNN blocks with Dual-Path RNN (DPRNN) blocks. The overall design remains similar to DeepFilterNet, but the dual-path structure allows separate modeling of temporal and frequency dependencies.

A similar idea is used in Hierarchical DFNet [4], where the model is split into two components: a coarse spectral enhancement network and a temporal enhancement network. The authors achieve a very low parameter count while maintaining competitive performance. This reflects a broader trend of replacing or restructuring recurrent components to improve efficiency.

2.4 Research Gap

Most existing works focus on modifying the overall architecture or improving the training procedure. However, relatively little attention is given to improving the efficiency of recurrent blocks inside such models.

Some recent works, such as Fast ULCNet [2], explore the use of more efficient recurrent architectures for speech enhancement and show that significant parameter reduction is possible. However, such approaches are not studied in the context of DeepFilterNet and similar hybrid architectures.

In particular, the effect of replacing standard GRU units with more parameter-efficient alternatives in DeepFilterNet remains underexplored. Additionally, there is limited analysis of how parameter allocation between different parts of the model affects performance.

In this work, we address these questions by studying the use of FastGRNN-based units in DeepFilterNet and by analyzing the effect of rebalancing parameters between the encoder-decoder and the deep filtering components.

Table 2.1: Comparison of named models. Dataset: VCTK+DEMAND. Metrics were adopted from [4], [6].

	Para.(M)	MACs(G/s)	PESQ	STOI	CSIG	CBAK	COVL
RNNoise	0.06	0.04	2.34	88.5	3.40	2.51	2.84
DeepFilterNet	1.78	0.35	2.81	-	4.14	3.31	3.46
HDF-Net	0.20	0.43	3.01	-	4.24	3.52	3.64
DPDFNet	3.54	4.37	2.90	93.4	-	-	-

3 Methodology

3.1 Signal model

Let $s(t) \in \mathbb{R}^{N \times T}$ be the clean speech signal, where N is the number of channels and T is the number of time stamps. The noisy observation $x(t)$ can be represented as

$$x = s * h + \epsilon,$$

where h is the room impulse response from the speaker to the microphone and ϵ is additive noise.

We operate in the short-time Fourier transform (STFT) domain since spectral transforms are an expressive feature transform allowing for smaller parameter counts. Under a standard narrowband approximation, the model can be written as

$$X(k, f) = S(k, f)H(k, f) + N(k, f),$$

where $S(k, f)$ is the STFT of the clean speech, $N(k, f)$ is the STFT representation of the additive noise, k is the time-frame index, and f is the frequency-bin index. The same notation applies to $X(k, f)$, $H(k, f)$, and $N(k, f)$.

3.2 Original DeepFilterNet architecture

Full framework of DeepFilterNet is illustrated in Fig. 3.1. We apply STFT to the noisy audio signal $x(n)$ and then observe two types of features. $X_{\text{ERB}}(k, b)$, where $b \in [0, \text{N}_{\text{ERB}}]$ and N_{ERB} is a number of bands. To extract ERB features, we first compute the STFT magnitude and convert it to log-power representation. Then, frequency bins are grouped into perceptual bands using an ERB filter bank.

The resulting features are normalized using an exponential moving average over time. This provides a compressed and stable representation of the spectrum, reducing dimensionality while

preserving important speech information, which helps the model focus on relative spectral changes rather than absolute signal level.

The second type of features DeepFilterNet uses is standard complex spectrogram X_{DF} . An exponential unit normalization is applied. Complex features are passed to the deep filter network.

DeepFilterNet follows an encoder-decoder architecture to predict ERB-domain gains. These gains are then mapped back to the frequency domain using an inverse ERB filter bank and applied to the noisy spectrogram by pointwise multiplication.

To further refine the result, the model predicts complex-valued deep filtering coefficients of order N . These coefficients are applied in the STFT domain and help improve periodic components of speech. In practice, deep filtering is only used up to a cutoff frequency f_{DF} , since most periodic speech energy is concentrated in lower frequencies.

The neural network itself is designed to be efficient and uses only standard building blocks such as convolutions, batch normalization, and ReLU activations. The authors suggest adopting a U-Net-like architecture with encoder and decoder pathways connected by skip connections, which helps preserve frequency resolution.

The convolutional blocks are based on separable convolutions (depthwise followed by 1×1 convolution). The layers are arranged in time such that only the first layers introduce a small look-ahead l_{DNN} , while the remaining layers are causal and do not increase latency.

For the recurrent and linear layers, grouped computation is used to reduce complexity. The input is split into P groups, processed independently, and then combined back. This allows the model to maintain a relatively large hidden size while keeping the computational cost low.

Finally, additional skip connections are used to preserve information about the original noisy phase, which improves the quality of the reconstructed signal.

3.3 FastGRNN block

Our main approach is to replace GRU blocks with Comfi-FastGRNN blocks that were proposed in [2]. This allows us to lower the number of parameters without a significant drop in performance. The architecture of the block is shown in Fig. 3.2.

FastGRNN uses a gated update with a residual connection. The hidden state is updated

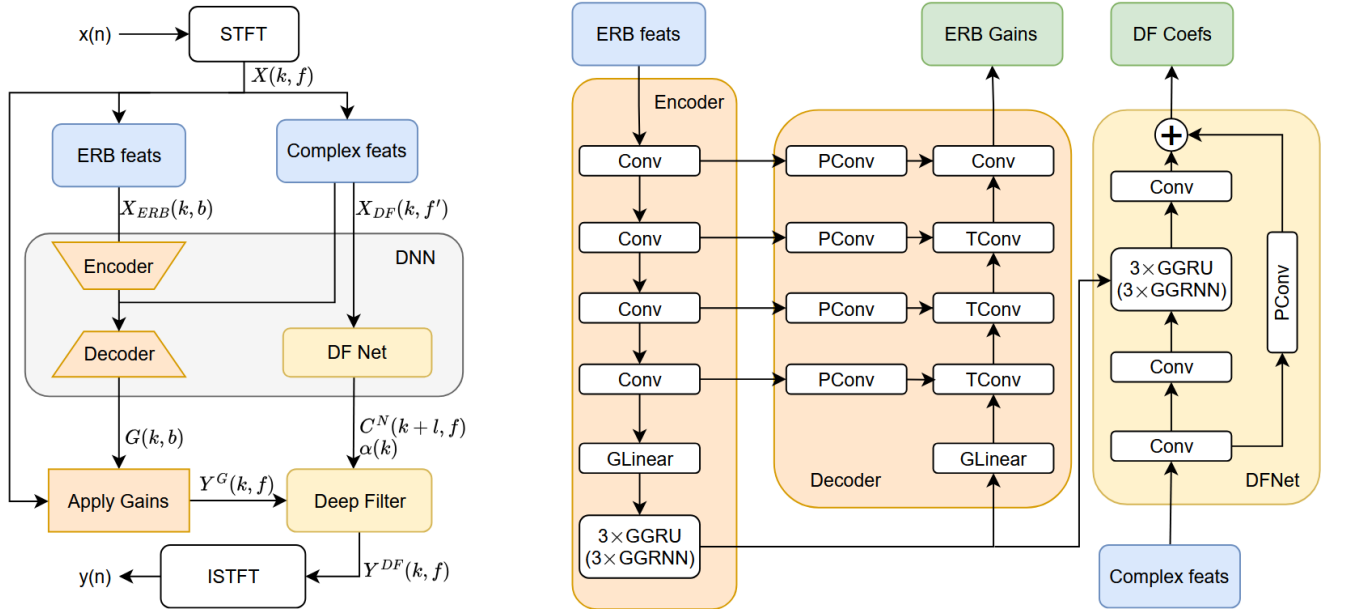


Figure 3.1: This illustration shows full DeepFilterNet framework. GLinear and GGRU (GGRNN) refer to the grouped blocks of Linear, GRU (FastGRNN) layers. Inspired by [11].

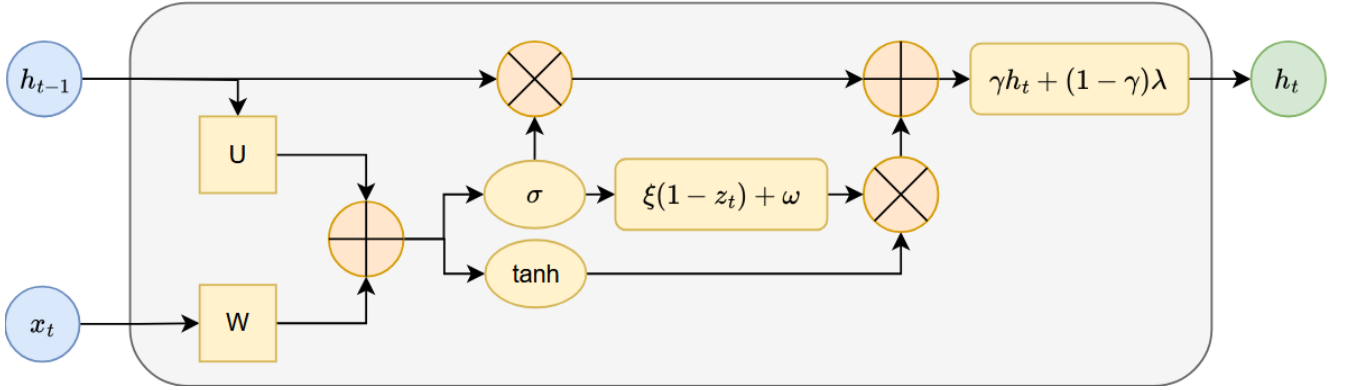


Figure 3.2: This illustration shows full Comfi-FastGRNN block. Inspired by [2].

as

$$z_t = \sigma(Wx_t + Uh_{t-1} + b_z), \quad (1)$$

$$\tilde{h}_t = \tanh(Wx_t + Uh_{t-1} + b_h), \quad (2)$$

$$h_t = (\xi(1 - z_t) + \omega) \odot \tilde{h}_t + z_t \odot h_{t-1}, \quad (3)$$

where $\xi, \omega \in [0, 1]$ are learnable scalar parameters. This design reduces the number of parameters and stabilizes training.

However, when applied to long sequences, FastGRNN may suffer from drift in the hidden state during inference. To address this, ComFi-FastGRNN introduces a correction term and modifies the update as

$$h_t^{\text{comfi}} = \gamma h_t + (1 - \gamma)\lambda, \quad (4)$$

where γ controls the contribution of the original hidden state and λ is a learnable vector.

This modification helps to prevent accumulation of errors over time and results in more stable behavior, while keeping the model lightweight.

3.4 Convolutional blocks

Inspired by a recent paper, we also experiment with several types of convolutional blocks that have shown the best results in [9]. Still, we have to make some changes to adapt the blocks to DeepFilterNet framework.

First, we adapt all convolutional layers to match the original DeepFilterNet geometry. In particular, we use asymmetric kernels of the form (k_t, k_f) , where the frequency dimension is fixed to 3 when an integer kernel size is provided. This follows the convention used in DeepFilterNet and ensures compatibility with ERB features.

In addition, we control causality in convolutional layers using manual temporal padding. A look-ahead parameter defines the amount of future context and therefore the latency.

We use three types of blocks:

- **Conv:** standard convolution $\rightarrow$ BN $\rightarrow$ activation, used as a baseline block in [9].
- **DWS:** pointwise convolution followed by depthwise convolution for efficient channel and spatial processing [1].
- **MB:** inverted residual design introduced in MobileNetV2 [10].

All the architectures are illustrated in Fig. 3.3.

3.5 Affine PReLU

Affine PReLU We use Affine PReLU (APReLU) proposed in [9], which extends standard PReLU by adding a learnable affine transformation. The activation is defined as

$$h(x) = \gamma x + \beta + \text{PReLU}(x),$$

where γ and β are learnable parameters that depend on channel and frequency. This allows the model to adapt feature scaling across frequency bands, which is important for speech signals.

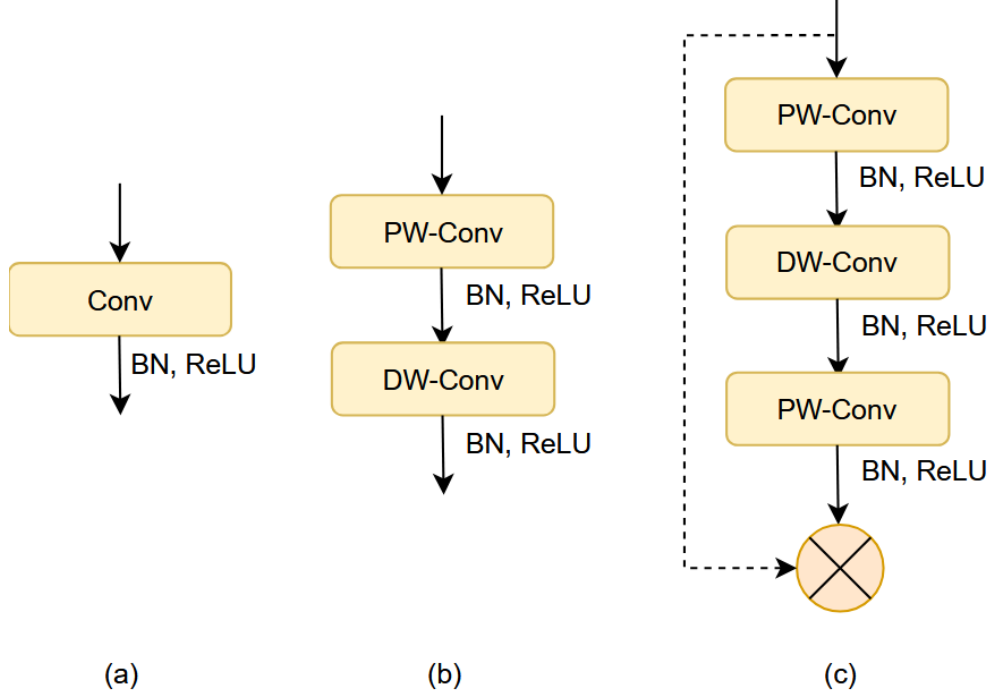


Figure 3.3: This illustration shows the architecture of the used blocks: (a) Conv (b) DWS (c) MB. Inspired by [9].

3.6 Loss

The total loss is defined as a combination of two terms:

$$\mathcal{L} = \lambda_{\text{spec}} \cdot \mathcal{L}_{\text{spec}}(Y, S) + \lambda_{\alpha} \cdot \mathcal{L}_{\alpha}, \quad (5)$$

where $\mathcal{L}_{\text{spec}}$ is a spectral reconstruction loss and $\mathcal{L}_{\alpha}$ is an additional term controlling the contribution of the deep filtering component.

Spectral loss The spectral loss is computed in the STFT domain and compares the enhanced signal Y with the clean signal S . It consists of two parts: a magnitude term and a phase-aware term:

$$\mathcal{L}_{\text{spec}} = \sum_{k,f} |||Y(k, f)|^c - |S(k, f)|^c||^2 + \sum_{k,f} |||Y(k, f)|^c e^{j\varphi_Y(k, f)} - |S(k, f)|^c e^{j\varphi_S(k, f)}||^2, \quad (6)$$

where c is a compression factor.

The first term compares magnitudes and encourages correct spectral envelope reconstruction. The second term additionally takes phase into account by comparing complex-valued spectra.

Angle-aware stabilization To improve stability for time–frequency bins with small magnitude, the phase term is computed using a normalized representation:

$$\frac{\partial \varphi}{\partial X} = \delta X \cdot \left(-\frac{\Im\{X\}}{|X_h|^2}, \frac{\Re\{X\}}{|X_h|^2} \right), \quad (7)$$

where $|X_h|^2 = \max(|X|^2, \varepsilon)$ is a stabilized magnitude.

Deep filtering control term The second part of the loss controls where deep filtering should be applied. It is defined as

$$\mathcal{L}_\alpha = \sum_k \|\alpha \cdot \mathbf{1}_{\text{LSNR} < -10 \text{ dB}}\|^2 + \sum_k \|(1 - \alpha) \cdot \mathbf{1}_{\text{LSNR} > -5 \text{ dB}}\|^2, \quad (8)$$

where LSNR is the local signal-to-noise ratio.

This term encourages the model to apply deep filtering mainly in regions where speech is present and avoids using it in noise-only segments.

3.7 Evaluation metrics

In addition to computational efficiency metrics such as parameter count and MACs (Multiply–Accumulate Operations), we use several quality metrics to evaluate our model. For 16 kHz speech enhancement, widely used objective metrics include PESQ (Perceptual Evaluation of Speech Quality) [7] and STOI (Short-Time Objective Intelligibility) [12].

PESQ measures perceived speech quality by comparing the enhanced signal with a clean reference. STOI focuses on intelligibility and shows how good speech content is after enhancement.

We also report SI-SDR (scale-invariant signal-to-distortion ratio). This metric measures similarity between enhanced and clean signals and is invariant to scaling, which makes it suitable for model comparison.

Using several metrics helps evaluate different aspects of performance, since improvements in one metric do not always lead to improvements in others.

We measure MACs using thop Python library.

4 Experimental setup

4.1 Datasets

Unlike the DeepFilterNet authors, we use the VCTK dataset [14] instead of the original DNS Challenge dataset [5]. In particular, we use a noisy speech database designed for training speech enhancement algorithms and TTS systems. The dataset is constructed using noise samples from the DEMAND dataset [13], therefore in most papers it is referred to as VCTK+DEMAND. It is widely used in speech enhancement research as a standard benchmark, which allows for direct comparison between different methods. For example, it is commonly used in works evaluating lightweight and real-time speech enhancement models.

The dataset consists of clean speech recordings from the VCTK corpus [17], which contains speech from multiple speakers with diverse accents, and corresponding noisy versions obtained by mixing with environmental noise. The noise signals are taken from the DEMAND dataset and include a variety of real-world acoustic conditions, such as domestic, office, and outdoor environments.

The major difference from the DNS dataset is that by choosing VCTK, we do not generate noisy recordings on-the-fly during training, but instead use pre-generated pairs of clean and noisy signals. This simplifies the training pipeline and ensures reproducibility of experiments, but at the same time reduces flexibility in terms of noise sampling and mixing strategies compared to DNS-based approaches.

The data is available in several sampling rates: 28 kHz, 48 kHz, and 56 kHz. However, we use the 48 kHz version and resample it to 16 kHz, since many evaluation metrics are defined for this sampling rate. In addition, working at 16 kHz reduces computational cost and is consistent with typical real-time deployment scenarios on resource-constrained devices.

We use the official train/test split. The training set consists of 11572 samples and the evaluation set consists of 824 samples.

4.2 Baselines

Original DeepFilterNet. We ran the original DeepFilterNet pipeline on our evaluation set.

However, we cannot directly compare our results with the original paper, since the training data differs significantly and the training setup is not the same in terms of both data generation and computational budget.

Therefore, this experiment serves as a reference point rather than a strict baseline. The main comparisons in this work are performed with our own implementation of the model. Still, it is useful to see how the training setup affects the metrics

Our implementation of DeepFilterNet. Since the authors of the original paper later focused on frameworks for hearing aid devices, their public implementation includes several optimizations, such as a custom Rust-based framework for the training process (dataloader, datasets, etc.).

Our implementation is done in PyTorch and is fully written in Python. All further baselines are also implemented within our template, so the main comparison is performed with this model.

This version is trained from scratch on the VCTK+DEMAND dataset and evaluated on the official evaluation set. All hyperparameters are kept the same as in the original DeepFilterNet. This ensures that differences in performance are mainly due to the implementation and training setup rather than hyperparameter changes.

FastDeepFilterNet. For this baseline, we replace GRU blocks with Comfi-FastGRNN units. To the best of our knowledge, this is one of the first works in speech enhancement that explores the use of FastGRNN in this setting, which makes this experiment relevant.

We keep the original dimensions of DeepFilterNet and expect to reduce the number of parameters due to the more efficient structure of Comfi-FastGRNN, while keeping the metrics at approximately the same level.

We evaluate whether replacing standard recurrent units with more parameter-efficient ones can improve the overall trade-off between model size and performance.

At the same time, this modification may affect temporal modeling, which we evaluate in the experiments.

Rebalanced DeepFilterNet. We train three different versions of a rebalanced model. The key idea here is to concentrate more parameters in the deep filtering part rather than in the encoder-decoder. By this shift of focus, we aim to achieve higher metrics with approximately the same number of parameters.

The intuition behind these experiments is that the deep filtering stage is responsible for denoising and reconstruction of harmonic structure. Therefore, increasing its capacity may lead to better performance, especially in STOI and PESQ metrics, which reflect perceived speech quality and intelligibility.

At the same time, reducing the capacity of the encoder-decoder may negatively affect feature extraction quality, which can limit the overall performance of the model.

Table 4.1 shows the configurations of our experiments. The models are numbered from 1

to 3 based on how aggressive the rebalancing is (higher numbers correspond to a stronger shift of parameters towards the deep filtering part).

Table 4.1: Concrete hyperparameters for rebalanced models.

	ch_{conv}	H_{emb}	N_{emb}	H_{df}	N_{df}	Params(M)
Original	64	512	3	512	2	1.69
Rebalanced-1	48	512	3	576	2	1.03
Rebalanced-2	40	512	3	640	2	1.09
Rebalanced-3	32	512	3	704	2	1.15

Convolutional blocks experiments. We also compare different convolutional blocks. The baselines are named DWS, MB and Conv for basic versions, and XDWS, XMB, XConv for versions with APReLU. We integrate all these blocks into the FastDeepFilterNet model that uses FastGRNN modules.

The goal is to evaluate the trade-off between computational efficiency and model performance. In particular, we study whether these blocks can improve quality at low additional cost.

4.3 Training

The training setup is the same for all baselines. We use the AdamW optimizer with a learning rate of $1e-3$. A StepLR scheduler is applied with $\gamma = 0.9$. One epoch corresponds to a full pass over the training set with a batch size of 32. Zero-padding is applied using the input padding mode.

The model operates in the STFT domain. We use a sampling rate of 16 kHz with an FFT size of 320 and a hop length of 160, corresponding to a 50% overlap. For ERB features, we use 24 bands, while the deep filtering stage operates on 96 frequency bins.

Feature normalization is performed using an exponential moving average with time constant $\tau = 1$. The local SDR is clipped to the range $[-15, 35]$ dB to stabilize training. The deep filtering module uses a filter order of 5 with a look-ahead of 1 frame.

You can find all the configurations in our official repository.

4.4 Results

4.4.1 Our models

Our implementation of DeepFilterNet serves as the main internal baseline for all further experiments. This confirms that our implementation is valid and suitable as a baseline. Direct

Table 4.2: Results of evaluating the models on the VCTK+DEMAND dataset. Best results are shown in bold, second-best are underlined.

	Params (M) ↓	MACs (G) ↓	PESQ ↑	STOI ↑	SI-SDR ↑
Original DFNet	1.78	0.35	2.81	0.94	16.63
Our DFNet	1.69	0.51	<u>2.31</u>	<u>0.93</u>	17.04
FastDFNet	<u>1.04</u>	0.34	2.26	<u>0.93</u>	<u>17.00</u>
Rebalanced-1	1.03	0.25	2.21	0.92	16.53
Rebalanced-2	1.09	<u>0.21</u>	2.24	0.92	16.83
Rebalanced-3	1.15	0.19	2.16	0.92	16.49
DWS	<u>1.04</u>	<u>0.32</u>	2.25	<u>0.93</u>	16.86
MB	1.08	0.52	2.24	<u>0.93</u>	16.96
Conv	1.11	0.68	2.25	<u>0.93</u>	16.97
XDWS	1.11	<u>0.32</u>	2.24	0.92	16.71
XMB	1.17	<u>0.50</u>	2.25	<u>0.93</u>	16.79
XConv	1.12	0.68	2.26	<u>0.93</u>	16.77

comparison with the official implementation is limited, as discussed above. The difference in MACs is likely due to differences in measurement methodology and implementation details.

The results are shown in Table 4.2. Replacing GRU blocks with Comfi-FastGRNN leads to a significant reduction in the number of parameters (from 1.69M to 1.04M), while the drop in metrics is relatively small. In particular, STOI remains unchanged, and the decrease in PESQ and SI-SDR is limited. We can conclude that FastGRNN is an effective lightweight alternative to GRU, although it slightly reduces the model’s ability to capture temporal dependencies.

The rebalanced models do not show improvement in metrics despite reallocating parameters towards the deep filtering stage. As the rebalancing becomes more aggressive, performance gradually decreases, which indicates that increasing the capacity of the deep filtering component alone is not sufficient to improve overall quality.

The reason may be that the encoder–decoder part of the model plays an important role in extracting meaningful spectral features. Reducing its capacity leads to weaker representations, which cannot be compensated for by a more powerful deep filtering stage.

At the same time, we observe a consistent reduction in computational cost for the rebalanced models. MACs decrease as more parameters are shifted towards the deep filtering part. Although the rebalancing does not improve quality metrics, it provides a more efficient model in terms of computation.

Both experiments confirm that changing the architecture without carefully preserving the balance between representation learning and signal reconstruction may lead to performance degradation.

Finally, we analyze the effect of convolutional blocks. DWS provides the best efficiency

with lower MACs, while MB and Conv offer slightly better quality at higher cost. This indicates that convolutional design plays a secondary role compared to recurrent modeling in DeepFilterNet.

The use of Affine PReLU (X-blocks) does not lead to consistent improvements. It shows that activation modification alone is not sufficient in this setup.

Overall, the best trade-off between quality and efficiency is achieved by FastDFNet and DWS-based variants. This confirms that lightweight architectural changes are more effective than increasing model complexity for this task.

4.4.2 Comparison with other models

Table 4.3: Performance comparison on the VCTK+DEMAND dataset. Best results are shown in bold, second-best are underlined. SI-SDR is used as an approximation of SI-SNR for comparison. The metrics were adopted from [9].

	Params (M) ↓	MACs (G) ↓	PESQ ↑	STOI ↑	SI-SDR ↑
Noisy	-	-	1.97	0.921	8.45
RNNoise	<u>0.06</u>	<u>0.04</u>	2.29	-	-
PercepNet	8.00	0.80	2.73	-	-
FullSubNet	5.64	30.87	2.89	0.940	-
DeepFilterNet	1.80	0.35	2.81	0.942	16.63
GaGNet	5.95	1.65	2.94	0.940	-
FastFullSubNet	6.84	4.14	2.79	0.935	-
GTCRN	<u>0.05</u>	0.03	2.87	0.940	18.83
LiSenNet	0.04	0.06	2.95	0.937	-
LiSenNet (+PESQ)	0.04	0.06	<u>3.07</u>	0.939	-
UL-UNAS	0.17	0.03	2.96	<u>0.941</u>	18.25
UL-UNAS (+PESQ)	0.17	0.03	3.09	<u>0.941</u>	<u>18.48</u>
Our DFNet	1.69	0.51	2.31	0.934	17.04
FastDFNet	1.04	0.34	2.26	0.924	17.00
Rebalanced-1	1.03	0.25	2.21	0.921	16.53
Rebalanced-3	1.15	0.19	2.16	0.922	16.49
DWS	1.04	0.32	2.25	0.928	16.86

We also provide a comparison with other models in Table 4.3. The results show that modern lightweight architectures significantly outperform DeepFilterNet in terms of both efficiency and quality. In particular, UL-UNAS achieves the best PESQ (3.09) while maintaining very low computational cost (0.03 MACs), and GTCRN provides the best SI-SDR (18.83) with extremely small model size.

Although our models remain competitive within the DeepFilterNet family, there is still a noticeable gap between our models and state-of-the-art methods.

The results confirm that while DeepFilterNet can be made more efficient, but achieving state-of-the-art performance requires more fundamental architectural changes.

5 Plan of further work

Our plans are to continue experimenting with different architectural choices.

First of all, we want to evaluate the use of GTCRN – another RNN-based architecture that was introduced in [8] and is designed as a part of an ultra-lightweight model for speech enhancement. The authors report high-quality results with only 27K parameters. However, we need to reproduce the model and integrate it into our training pipeline before making a fair comparison.

Secondly, there are a number of works that explore hierarchical versions of DeepFilterNet like [4]. We plan to combine such architectures with more efficient recurrent blocks, such as CombiFastGRNN or GTCRN, to further study the trade-off between model capacity and performance.

Another direction of research is the use of the DNS 4 Challenge dataset, which is widely adopted in the speech enhancement community. Evaluation on this dataset would allow for a more direct comparison with many recent models that do not provide open-source implementations. However, at the current stage, such experiments are limited by computational and storage constraints, as the dataset exceeds the available resources on the cCHARISMA supercomputer. We have already submitted a request to add the DNS dataset to list of available datasets.

Finally, we plan to explore the use of GAN-based modules to improve perceptual quality. In particular, adversarial training may help the model better capture fine-grained structure of speech and reduce artifacts that are not well addressed by standard spectral losses. This could lead to improvements in perceptual metrics such as PESQ, even if signal-level metrics remain similar. At the same time, such approaches require careful balancing of the loss terms and may increase training instability, which we plan to investigate.

6 Conclusion

In this work, we explored several lightweight modifications of DeepFilterNet. Replacing GRU with FastGRNN significantly reduces the number of parameters while keeping similar quality. This makes it a good option for resource-limited settings.

Rebalancing parameters towards the deep filtering part reduces computation but does not improve performance. This suggests that the original balance between model components is already close to optimal.

Experiments with convolutional blocks show that changing local convolution structure has limited effect. The use of APreLU also does not give consistent improvements.

Overall, DeepFilterNet can be made more efficient with simple changes. However, to reach state-of-the-art results, more significant changes in the model design are needed.

Use of AI Tools

AI tools (ChatGPT, GPT-5.3) were used to help with phrasing and as a better version of translator. At early stages, they were also used to better understand the topic and related papers.

References

- [1] Andrew Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications”. In: (Apr. 2017). DOI: [10.48550/arXiv.1704.04861](https://doi.org/10.48550/arXiv.1704.04861).
- [2] Nicolás Larraza and Niels Koeijer. *Fast-ULCNet: A fast and ultra low complexity network for single-channel speech enhancement*. Jan. 2026. DOI: [10.48550/arXiv.2601.14925](https://doi.org/10.48550/arXiv.2601.14925).
- [3] Philipos C. Loizou. *Speech Enhancement: Theory and Practice*. 2nd ed. Boca Raton, FL: CRC Press, 2013. ISBN: 9781466504219. DOI: [10.1201/b14529](https://doi.org/10.1201/b14529).
- [4] Shenghui Lu, Hukai Huang, Jinanglong Yao, Kaidi Wang, Qingyang Hong, and Lin Li. *A Two-Stage Hierarchical Deep Filtering Framework for Real-Time Speech Enhancement*. June 2025. DOI: [10.48550/arXiv.2506.01023](https://doi.org/10.48550/arXiv.2506.01023).
- [5] Chandan KA Reddy, Vishak Gopal, Ross Cutler, Ebrahim Beyrami, Roger Cheng, Harishchandra Dubey, Sergiy Matuskevych, Robert Aichner, Ashkan Aazami, Sebastian Braun, et al. “The INTERSPEECH 2020 deep noise suppression challenge: Datasets, subjective testing framework, and challenge results”. In: *INTERSPEECH*. 2020.
- [6] Daniel Rika, Nino Sapir, and Ido Gus. *DPDFNet: Boosting DeepFilterNet2 via Dual-Path RNN*. Dec. 2025. DOI: [10.48550/arXiv.2512.16420](https://doi.org/10.48550/arXiv.2512.16420).
- [7] A. W. Rix, J. G. Beerends, M. P. Hollier, and A. P. Hekstra. “Perceptual evaluation of speech quality (PESQ): A new method for speech quality assessment of telephone networks and codecs”. In: *Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*. Vol. 2. 2001, pp. 749–752. DOI: [10.1109/ICASSP.2001.941023](https://doi.org/10.1109/ICASSP.2001.941023).
- [8] Xiaobin Rong, Tianchi Sun, Xu Zhang, Yuxiang Hu, Changbao Zhu, and Jing Lu. “GTCRN: A Speech Enhancement Model Requiring Ultralow Computational Resources”. In: *ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 2024, pp. 971–975. DOI: [10.1109/ICASSP48485.2024.10448310](https://doi.org/10.1109/ICASSP48485.2024.10448310).
- [9] Xiaobin Rong, Leyan Yang, Dahan Wang, Yuxiang Hu, Changbao Zhu, Kai Chen, and Jing Lu. “UL-UNAS: Ultra-Lightweight U-Nets for Real-Time Speech Enhancement via Network Architecture Search”. In: *IEEE Transactions on Audio, Speech and Language Processing* 34 (2026), pp. 1085–1096. DOI: [10.1109/TASLPRO.2026.3661271](https://doi.org/10.1109/TASLPRO.2026.3661271).

- [10] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. “MobileNetV2: Inverted Residuals and Linear Bottlenecks”. In: *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2018, pp. 4510–4520. DOI: [10.1109/CVPR.2018.00474](https://doi.org/10.1109/CVPR.2018.00474).
- [11] Hendrik Schröter, Alberto N. Escalante B., Tobias Rosenkranz, and Andreas Maier. *Deep-FilterNet: A Low Complexity Speech Enhancement Framework for Full-Band Audio based on Deep Filtering*. Oct. 2021. DOI: [10.48550/arXiv.2110.05588](https://doi.org/10.48550/arXiv.2110.05588).
- [12] Cees H. Taal, Richard C. Hendriks, Richard Heusdens, and Jesper Jensen. “An Algorithm for Intelligibility Prediction of Time–Frequency Weighted Noisy Speech”. In: *IEEE Transactions on Audio, Speech, and Language Processing* 19.7 (2011), pp. 2125–2136. DOI: [10.1109/TASL.2011.2114881](https://doi.org/10.1109/TASL.2011.2114881).
- [13] Joachim Thiemann, Nobutaka Ito, and Emmanuel Vincent. “The Diverse Environments Multi-channel Acoustic Noise Database (DEMAND): A database of multichannel environmental noise recordings”. In: *Proceedings of the 21st International Congress on Acoustics (ICA 2013)*. HAL preprint: hal-00796707. Montreal, Canada: Acoustical Society of America, June 2013. DOI: [10.5281/zenodo.1227120](https://doi.org/10.5281/zenodo.1227120).
- [14] Cassia Valentini-Botinhao. *Noisy speech database for training speech enhancement algorithms and TTS models*. Dataset. 2017. DOI: [10.7488/ds/2117](https://doi.org/10.7488/ds/2117). URL: <https://doi.org/10.7488/ds/2117>.
- [15] Jean-Marc Valin. “A Hybrid DSP/Deep Learning Approach to Real-Time Full-Band Speech Enhancement”. In: Aug. 2018. DOI: [10.1109/MMSP.2018.8547084](https://doi.org/10.1109/MMSP.2018.8547084).
- [16] Shilin Wang, Haixin Guan, Shuang Wei, and Yanhua Long. “Improving low-complexity and real-time DeepFilterNet2 for personalized speech enhancement”. In: *International Journal of Speech Technology* 27 (Apr. 2024), pp. 1–8. DOI: [10.1007/s10772-024-10101-z](https://doi.org/10.1007/s10772-024-10101-z).
- [17] Junichi Yamagishi, Christophe Veaux, and Kirsten MacDonald. *CSTR VCTK Corpus: English Multi-speaker Corpus for CSTR Voice Cloning Toolkit*. Version 0.92. Sound dataset. 2019. DOI: [10.7488/ds/2645](https://doi.org/10.7488/ds/2645). URL: <https://doi.org/10.7488/ds/2645>.